Practical:05	Implementation of a knapsack problem using dynamic programming.
__/___/2026	

Objective:

To solve the 0/1 Knapsack problem using Dynamic Programming to determine the maximum total value obtainable without exceeding a given knapsack weight capacity.

Program:

```
#include <iostream>
#include <vector>
using namespace std;

int knapsack(int capacity, vector<int>& weight, vector<int>& value, int n)
{
    // DP table
    vector<vector<int>> dp(n + 1, vector<int>(capacity + 1, 0));

    // Build the DP table
    for (int i = 1; i <= n; i++)
    {
        for (int w = 1; w <= capacity; w++)
        {
            // If current item can fit
            if (weight[i - 1] <= w)
            {
                dp[i][w] = max(
                    value[i - 1] + dp[i - 1][w - weight[i - 1]],
                    dp[i - 1][w]
                );
            }
            else
            {
                // Cannot include current item
```

```
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    return dp[n][capacity];
}

int main()
{
    int n, capacity;

    cout << "Enter number of items: ";
    cin >> n;

    vector<int> weight(n);
    vector<int> value(n);

    cout << "Enter weights of items:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> weight[i];
    }

    cout << "Enter values of items:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> value[i];
    }

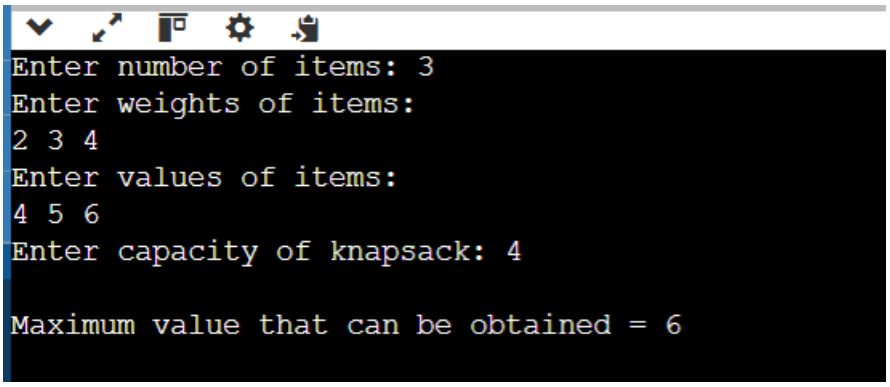
    cout << "Enter capacity of knapsack: ";
    cin >> capacity;
```

```
    int maxValue = knapsack(capacity, weight, value, n);

    cout << "\nMaximum value that can be obtained = "
         << maxValue << endl;

    return 0;
}
```

Output:



Analysis:

a) Problem Analysis

The 0/1 Knapsack Problem is an optimization problem in which we have:

- n items
- Each item has a weight
- Each item has a value
- A knapsack with a maximum capacity

The objective is to select items such that:

1. Total weight does not exceed the knapsack capacity.
2. Total value is maximum.
3. Each item can be selected only once.

b) Dynamic Programming Approach

We create a 2D table dp[i][w].

Here:

- i = number of items considered
- w = current knapsack capacity

- dp[i][w] = maximum value that can be obtained using the first i items with capacity w.

For every item, we have two choices:

Case 1: Include the item

If the item's weight is less than or equal to the current capacity:

value[i-1] + dp[i-1][w-weight[i-1]]

Case 2: Exclude the item

dp[i-1][w]

We choose the maximum of these two:

dp[i][w] = max(

value[i-1] + dp[i-1][w-weight[i-1]],

dp[i-1][w]

)

If the item cannot fit:

dp[i][w] = dp[i-1][w]

c) Example Analysis

Consider:

Number of items = 4

Capacity = 5

Weights = 2 3 4 5

Values = 3 4 5 6

Possible combinations:

Items Selected	Total Weight	Total Value
Item 1	2	3
Item 2	3	4
Item 3	4	5
Item 4	5	6
Item 1 + Item 2	5	**7**

Since the capacity is 5, the combination **Item 1 + Item 2** gives the highest value.

Therefore:

Maximum Value = 7

d) Algorithm Analysis

Step 1: Read the number of items.

Step 2: Read the weight and value of each item.

Step 3: Read the capacity of the knapsack.

Step 4: Create a DP table of size:

(n + 1) × (capacity + 1)

Step 5: Initialize the first row and first column with 0.

Step 6: For every item and every possible capacity:

- Check whether the item can fit.
- If it can fit, calculate both include and exclude cases.
- Store the maximum value.
- Otherwise, exclude the item.

Step 7: The final answer is stored at:

dp[n][capacity]

e) Time Complexity

There are n rows and W columns in the DP table.

Therefore:

Time Complexity = O(n × W)

where:

- n = number of items
- W = knapsack capacity

f) Space Complexity

The program uses a 2D DP table:

dp[n+1][W+1]

Therefore:

Space Complexity = O(n × W)

Conclusion:

The 0/1 Knapsack problem was successfully implemented using Dynamic Programming. The DP approach divides the problem into smaller subproblems and stores their solutions to avoid repeated calculations. For the given example, the maximum value obtained is 7, with a total weight of 5. The algorithm has a time complexity of O(n × W) and a space complexity of O(n × W).